Reference vs. Value

There are two ways to pass data into a function, depending on whether or not you wish the incoming data to be altered. By default, PHP passes values into a function *by value* – this means that any changes made to the incoming argument data will not affect the value outside of the function scope. Consider the following code:

```
function addCar ($inputArr) {
    $inputArr[] = 'Scirocco';
}

$cars = ['Golf', 'Polo'];
addCar($cars);

print_r($cars);
```

On first glance it looks as if this might put an additional element into the `$cars` array. In fact, this does happen in a way, just not to the original array. Instead, PHP will create a copy of the `$cars` array and add the new element there. PHP internally will use the same data (*zval*) for both variables unless `addCar()` makes any changes to it. If that happens, then PHP creates a copy, the function block works with the copy, and the original array declared outside the function remains unchanged. This behavior is an optimization feature known as *copy-on-write*. The output of the above code is as follows:

```
Array
(
    [0] => Golf
    [1] => Polo
)
```

To have the new element added to the global `$cars` array, we need to make it clear to the PHP interpreter that we don't want this copy-on-write behavior. To achieve this, we use a special reference operator `&` immediately before the `$inputArr` argument in the function declaration. Let's change our code accordingly:

```
function addCar (&$inputArr) {
    $inputArr[] = 'Scirocco';
}
```